@suisei/sui-skills-mcp v0.1.0

Sui Skills MCP

The Sui Stack as one-job-per-tool primitives, over the Model Context Protocol.

Capability Reference · 19 tools · Generated 2026-05-28

This MCP server is the product: plug it into Claude Desktop, Cursor, Windsurf, or any MCP-aware agent and you get one-line tools for reading chain state, building any Sui transaction, simulating it, submitting it, and storing data on Walrus. **Suisei** — the Sui teaching agent — is a showcase of it: a working app built entirely on these tools, demonstrating what an agent can do once it speaks Sui.

1. What it is

Most Sui SDKs assume you are writing *application* code. This package assumes you are writing *agent* code: short tool calls, structured JSON in, structured JSON out, no React, no UI. The agent decides what to do; the tool does exactly one thing and returns machine-readable output.

The server speaks JSON-RPC 2.0 over **stdio** per the MCP specification. There is no network listener — the host agent process spawns the binary, talks to it over stdin/stdout, and supervises its lifecycle.

2. Design principles

Principle	What it means in practice
No private keys	Tools that produce transactions return base64 transaction bytes. The host signs and submits. A tool that holds a key is a tool that can spend money — so this toolkit never holds one.
One job per tool	No mega-tools with twelve flags. Each tool is unambiguous, which sharpens the calling agent's reasoning about when to use it.
Structured output	Every successful return is a JSON string in the text content block. Agents parse structured data far more reliably than prose.

3. The build loop

Write operations follow a four-step, non-custodial loop. The toolkit covers steps 1, 2 and 4; the host owns the key and performs step 3.

```
1. *_build tool → returns base64 unsigned tx bytes
2. sui_dry_run → simulate: status + gas, no spend
3. host signs the bytes (outside the toolkit — the host holds the key)
4. sui_execute_signed_tx → submit signed bytes, return digest + effects
```

> **Security invariant:** every transaction-building tool ends by returning bytes, never by signing. The private key never enters the MCP process.

4. Install & connect

```
npm install -g @suisei/sui-skills-mcp
```

Then register it with an MCP host. For Claude Desktop, edit `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "sui": {
      "command": "npx",
      "args": ["-y", "@suisei/sui-skills-mcp"]
    }
  }
}
```

Restart the host. All tools are exposed under the `sui` server namespace.

5. Tool reference

Nineteen tools across four groups. Every tool accepts a `network` argument (testnet · mainnet · devnet, default testnet) except the two Walrus tools, which target Walrus endpoints directly.

5.1 Read the chain

Tool	What it does
sui_resolve_address	SuiNS name → 0x address (idempotent on 0x input)
sui_get_balance	SUI balance in MIST + human-readable SUI
sui_get_all_balances	Every coin balance a wallet holds, not just SUI
sui_get_object	Any object's type, owner, version, fields, and Display
sui_get_owned_objects	List objects by owner, filter by struct type, paginated
sui_get_owned_badges	List a wallet's Suisei quest-completion badges
sui_get_coins	List coin objects of a type (the ids you spend), paginated
sui_get_transaction	Look up a finalized tx by digest: status, gas, changes
sui_get_reference_gas_price	Current network reference gas price (MIST)
sui_get_dynamic_fields	List an object's dynamic fields (Tables, Bags)

sui_resolve_address

If given a 0x address, returns it canonicalized (lowercased). Otherwise treats the input as a SuiNS name and resolves it. Use this whenever a user gives a human-readable name.

Input	Type	Req	Notes
name_or_address	string	yes	A SuiNS name or 0x address
network	enum	no	default testnet

```
→ { "address": "0x…", "source": "suins", "name": "alice.sui" }
```

sui_get_balance / sui_get_all_balances

`sui_get_balance` returns the native SUI balance for one address; `sui_get_all_balances` returns every coin type held. Both take address + network.

```
→ { "coin_type": "0x2::sui::SUI", "total_mist": "1500000000",
"total_sui": 1.5, "coin_object_count": 3 }
```

sui_get_object

The general read primitive — every other read tool is a special case of it. Returns Move type, owner, version, digest, content fields, and Display metadata for any object id.

```
→ { "type": "0x…::badge::Badge", "owner": {…}, "version": "…",
"fields": {…}, "display": {…} }
```

sui_get_owned_objects

Lists objects owned by an address, optionally filtered to a single fully-qualified Move struct type. Paginated — pass the returned next_cursor to continue.

Input	Type	Req	Notes
address	string	yes	Wallet to list
struct_type	string	no	e.g. 0x2::coin::Coin<0x2::sui::SUI>
cursor	string	no	Pagination cursor
limit	number	no	1–50, default 50
network	enum	no	default testnet

sui_get_owned_badges

Lists Suisei badges (soulbound quest-completion NFTs) owned by an address — useful for showing curriculum progress. Resolves the badge package from the badge_package argument, the SUISEI_BADGE_PACKAGE env var, or a built-in per-network default.

```
→ { "count": 3, "badges": [ { "object_id": "…",
"quest_id": "zklogin", "quest_number": 1, "minted_at_ms": "…" } ] }
```

sui_get_coins

Unlike sui_get_balance (which sums), this returns the concrete coin_object_id values an agent needs to spend or split in a transaction. Defaults to native SUI; pass coin_type for any other coin. Paginated.

```
→ { "coin_type": "0x2::sui::SUI", "count": 4, "has_next_page": false,
"coins": [ { "coin_object_id": "…", "balance": "1000000000" }, … ] }
```

sui_get_transaction

Fetch a finalized transaction by digest — the same digest sui_execute_signed_tx returns. Returns status, gas used, balance changes, timestamp, and event count.

```
→ { "digest": "…", "status": "success", "timestamp_ms": "…",
"gas_used": {…}, "balance_changes": [...], "events_count": 1 }
```

sui_get_reference_gas_price / sui_get_dynamic_fields

sui_get_reference_gas_price returns the current network gas price in MIST (takes only network). sui_get_dynamic_fields lists the dynamic fields attached to a parent object — how Sui stores Tables, Bags, and other on-chain collections — returning each field's name, type, and child object id (paginated).

5.2 Build a transaction (returns unsigned bytes)

Tool	What it does
sui_move_call	Build a call to ANY Move entry function — the universal write
sui_transfer	Build a transfer of SUI and/or whole objects
sui_stake	Build a native staking delegation to a validator
sui_unstake	Build a withdraw-stake for a StakedSui object
sui_mint_badge	Build a Suisei completion-badge mint

sui_move_call — the universal write primitive

Builds an unsigned transaction calling any Move entry function. Arguments are encoded as strings so the schema stays flat and unambiguous:

- `object:<id>` — an owned or shared object by id
- `pure:<type>:<value>` — type is address, id, bool, string, or u8/u16/u32/u64/u128/u256

> Vector and nested-struct arguments are intentionally out of scope for the generic tool. Use a purpose-built tool (e.g. sui_transfer) for those, so the generic schema stays unambiguous for the calling agent.

Input	Type	Req	Notes
target	string	yes	0xpkg::module::function
type_arguments	string[]	no	generics, e.g. ["0x2::sui::SUI"]
arguments	string[]	no	encoded as above
sender	string	yes	0x address that will sign & pay
network	enum	no	default testnet

```
→ { "tx_bytes_base64": "…", "target": "0x…::m::f",
"sender": "0x…", "next_step": "Dry-run, then sign and submit." }
```

sui_transfer

Provide amount_mist to send SUI (split from the gas coin) and/or object_ids to send whole objects. At least one is required.

Input	Type	Req	Notes
sender	string	yes	Sends & pays
recipient	string	yes	Receives
amount_mist	string	no*	SUI in MIST (string, avoids precision loss)
object_ids	string[]	no*	Whole objects to transfer

* one of amount_mist or object_ids is required.

sui_stake / sui_unstake

sui_stake splits amount_mist from gas and delegates to a validator via 0x3::sui_system::request_add_stake; the StakedSui object lands in the sender once signed. sui_unstake withdraws a StakedSui by id via request_withdraw_stake, returning principal plus rewards.

Tool	Required inputs
sui_stake	sender, amount_mist, validator
sui_unstake	sender, staked_sui_id

sui_mint_badge

Builds a PTB that mints a Suisei completion badge to a recipient through the canonical badge module. Returns unsigned bytes; the caller signs and submits.

Input	Type	Req	Notes
recipient	string	yes	0x address receiving the badge
quest_id	string	yes	e.g. zklogin, sponsored
quest_number	number	yes	1–255
badge_package	string	yes	Move package with the badge module

5.3 Simulate & submit

Tool	What it does
sui_dry_run	Simulate built tx bytes (status + gas, no spend)
sui_execute_signed_tx	Submit host-signed bytes, return digest + effects

sui_dry_run

Simulates an unsigned transaction without spending gas, so an agent can verify a builder result before asking the host to sign. Takes tx_bytes_base64.

```
→ { "status": "success", "gas_used": {…}, "balance_changes": [...],
"object_changes_count": 2, "events_count": 0 }
```

sui_execute_signed_tx

Submits a host-signed transaction. Takes the same tx_bytes_base64 that were signed plus one or more base64 signatures. The toolkit holds no keys — signing happens in the host.

```
→ { "digest": "…", "status": "success", "gas_used": {…},
"balance_changes": [...], "events_count": 1 }
```

5.4 Walrus storage

Tool	Key inputs	Returns
walrus_publish	content, encoding (utf8/base64), epochs (default 5), publisher_url?	blob_id, status, size_bytes
walrus_fetch	blob_id, as (utf8/base64), aggregator_url?	content, encoding, size_bytes

6. Configuration

Variable	Purpose
SUISEI_BADGE_PACKAGE	Default badge package id for sui_get_owned_badges when not passed explicitly.
Walrus endpoints	Default to the public Walrus testnet publisher/aggregator; override per-call via publisher_url / aggregator_url.
Network	Defaults to testnet; pass network on any tool to switch.

7. Security model

- **Non-custodial by construction.** No tool accepts, stores, or derives a private key. Builder tools return unsigned bytes; only sui_execute_signed_tx touches a signature, and that signature is produced by the host.
- **No ambient network surface.** Stdio transport only — the server opens no port and accepts no inbound connections.
- **Verify before spend.** The documented loop routes every write through sui_dry_run before signing, so an agent can confirm gas and effects up front.